

A. Ζητούμενο

Μελετήστε τον σειριακό κώδικα στο αρχείο:

<https://mixstef.github.io/courses/parprog/lu-serial.c>

Ο κώδικας αυτός είναι μια σειριακή μορφή παραγοντοποίησης LU (LU decomposition/factorization) ενός τετραγωνικού πίνακα $N \times N$ από doubles.

Ο στόχος της εργασίας είναι να κατασκευάσετε 3 εκδοχές του προηγούμενου προγράμματος σε C:

1. Σειριακή υλοποίηση του ίδιου αλγορίθμου, χρησιμοποιώντας όμως διάσπαση σε tiles (cache blocking).
2. Παράλληλη μορφή του (1), χρησιμοποιώντας OpenMP for constructs.
3. Παράλληλη μορφή του (1), χρησιμοποιώντας OpenMP tasks.

Σε όλες τις περιπτώσεις πρέπει να ξεκινήσετε από τον προηγούμενο σειριακό κώδικα. Η μόνη παρέμβαση που μπορείτε να κάνετε είναι στα περιεχόμενα της συνάρτησης `your_lu()`. Για την επίλυση ακολουθήστε τα βήματα της επόμενης ενότητας.

B. Διαδικασία επίλυσης

Ακολουθήστε τα παρακάτω βήματα για τη δημιουργία των 3 εκδοχών κώδικα σε C. Οι ερωτήσεις (με αρίθμηση E.x) που τίθενται σε κάθε στάδιο θα πρέπει να απαντηθούν στη συνοδευτική αναφορά που θα παραδώσετε μαζί με τον κώδικα.

1. Μετατροπή σε σειριακό αλγόριθμο με tiles (blocked)

Αρχικά θα πρέπει να βρείτε τον τρόπο ώστε ο αρχικός αλγόριθμος να εκτελείται (πάντα σειριακά) με διαχωρισμό σε tiles μεγέθους $TILESIZE \times TILESIZE$. Για τις ανάγκες της εργασίας θεωρήστε ότι το N είναι ακέραιο πολλαπλάσιο του $TILESIZE$. Αυτό όχι μόνο βελτιώνει την τοπικότητα στην προσπέλαση δεδομένων στις κρυφές μνήμες (cache blocking) αλλά και θα χρειαστεί για τα επόμενα βήματα (2) και (3).

Σημ.: η χρήση εργαλείων TN στο σημείο αυτό μπορεί να βοηθήσει σημαντικά!

(E.1) Περιγράψτε τα στάδια του νέου αλγορίθμου: ποιο μέρος του πίνακα υπολογίζεται σε κάθε στάδιο;

(E.2) Παραθέστε τις αλληλεξαρτήσεις μεταξύ tiles στον νέο αλγόριθμο. Με ποια σειρά πρέπει να υπολογιστούν τα tiles; Ποια tiles εξαρτώνται από ποια;

2. Παράλληλη υλοποίηση με OpenMP for constructs

Με βάση τις απαντήσεις σας στα E.1-2, τροποποιήστε τον σειριακό αλγόριθμο με τα tiles, έτσι ώστε να υλοποιείται με OpenMP for constructs (δηλ. χωρίς τη χρήση tasks).

(E.3) Περιγράψτε τη λογική του νέου προγράμματος, και κυρίως πώς εξασφαλίζεται η τήρηση των αλληλεξαρτήσεων που αναφέρατε στο E.2.

(E.4) Αν χρησιμοποιείτε χαρακτηριστικά του OpenMP που δεν έχουν διδαχθεί στο μάθημα, περιγράψτε ποια είναι αυτά και ποιος ο λόγος που τα προσθέσατε.

3. Παράλληλη υλοποίηση με OpenMP tasks

Κατασκευάστε τον αντίστοιχο κώδικα C, χρησιμοποιώντας αυτή τη φορά OpenMP tasks.

(E.5) Πώς εξασφαλίζεται τώρα η τήρηση των αλληλεξαρτήσεων; Περιγράψτε τα σημεία στον κώδικα τα οποία εξασφαλίζουν την τήρηση της σωστής σειράς υπολογισμού μεταξύ tiles.

(E.6) Συγκρίνετε τη διαφορά μεταξύ των μηχανισμών τήρησης των αλληλεξαρτήσεων στα βήματα 2 και 3 (OpenMP for και OpenMP tasks). Θεωρητικά, ποιο σχήμα επιτρέπει μεγαλύτερο βαθμό παραλληλίας;

4. Σύγκριση απόδοσης

Συγκρίνετε τον χρόνο εκτέλεσης του φορτίου στον σειριακό και στις 3 δικές σας εκδοχές κώδικα για διάφορες τιμές του N στην περιοχή μεταξύ 500 και 2500. Χρησιμοποιήστε TILESIZE ίσο με 16 και 32 (φροντίστε σε κάθε περίπτωση το N να είναι ακέραιο πολλαπλάσιο του TILESIZE!).

(E.7) Παραθέστε τις αποδόσεις όλων των προγραμμάτων (χρόνους εκτέλεσης) για τις διάφορες τιμές N και TILESIZE. Σχολιάστε τη διαφορά απόδοσης σε κάθε περίπτωση.

5. Αναφορά στη χρήση TN

(E.8) Σε περίπτωση χρήσης εργαλείων TN περιγράψτε σε ποιο στάδιο χρησιμοποιήθηκαν και με ποιον τρόπο.

Γ. Παραδοτέο

i) Όταν έχετε ολοκληρώσει την επίλυση της εργασίας θα πρέπει να έχετε τρία αρχεία C, ένα για κάθε εκδοχή υλοποίησης.

ii) Ετοιμάστε αναφορά σε μορφή pdf με τις απαντήσεις σας στα ερωτήματα E.x των προηγούμενων ενοτήτων.

Ανεβάστε τα 3 αρχεία C και την αναφορά pdf σε ένα μοναδικό αρχείο zip στο [opencourses](#) (Εργασία 2) έως και τη **Παρασκευή 5/6**.

Βαθμολόγηση εργασίας

Κριτήριο	% βαθμού
----------	----------

Επιτυχής συγγραφή κώδικα (και στις τρεις εκδοχές) με ορθό υπολογισμό αποτελεσμάτων	40%
Πλήρεις και τεκμηριωμένες απαντήσεις των ερωτημάτων στην αναφορά	60%